

Documentación Proyecto 2 PLE

Se adjunta link de la gramática de la primera entrega en caso de que sea necesaria para la evaluación del presente proyecto: [☰ PLE Task1 Project1](#)

Cambios lógicos realizados a la gramática o anotaciones importantes por la transferencia a lenguaje Rascal

- Identifier
 - PDF: `letter (letter | digit | “-”)*`
 - Código: `lexical Identifier = Letter (Letter | Digit | “-”)* !>> [a-zA-Z0-9\-\] \ Keywords;`
 - El *negative lookahead* garantiza el escaneo total de la palabra, evitando que los nombres de las variables se partan a la mitad. Además, se sustrajo el conjunto de palabras reservadas (Keywords) para evitar posibles errores.
- Components
 - PDF: `component+`
 - Código: -
 - En los lugares donde se necesitaba más de un componente se usó el component necesitado más + o *.
- Dominio
 - PDF: `identifier (“->” identifier)+`
 - Código: `syntax Dominio = dominio: Identifier {“-\>” Identifier}+;`
 - En vez de () se usó {}, que es la notación de listas separadas nativas de Rascal para facilitar la posterior iteración y extracción de tipos en el AST.
- Quantifier Expr
 - PDF: `“(” quantifier identifier “in” identifier “.” quantifier-exp “)” | expr`
 - Código: `syntax QuantifierExp = quantifierexp: “(” Quantifier Identifier “in” Identifier “.” Expr “)”;`
 - Se eliminó el | Expr y se cambió el último quantifier-exp por Expr para dar resolución a un posible ciclo infinito por recursividad por izquierda. Además, la jerarquía de Expr ya engloba Quantifier-exp.
- Expr
 - PDF: `expr bin-op expr
| un-op expr
| “(“ expr “)”
| quantifier-exp
| atomic-exp`
 - Código: `syntax Expr
= exprUnary: UnOp Expr
| exprParen: “(” Expr “)”
| exprQuant: QuantifierExp`

```
| exprAtomic: AtomicExp  
> left exprBin: Expr BinOp Expr;
```

- Resolución de ambigüedad en operaciones continuas: La regla original era ambigua frente a expresiones secuenciales (ej. $a + b + c$). Se implementó la resolución de conflictos nativa de Rascal usando el operador de precedencia escalar ($>$) y la etiqueta de asociatividad (`left`) para aislar las operaciones binarias y garantizar un árbol de derivación único.
- Atomic-expr
 - PDF: `identifier+ | literal`
 - Código:

```
syntax AtomicExp  
= atomicexp: Identifier  
| atomicexpsimple: Literal;
```
 - Se quitó el `+` del `identifier` y se delegó la responsabilidad de agrupar múltiples identificadores a reglas superiores evitando de esta forma la fragmentación incorrecta de una palabra.
- Attr-Item
 - PDF: `identifier (":" (identifier | expr))?`
 - Código:

```
syntax AttrItem = attrItem: Identifier (":" Expr)?;
```
 - Resolución de ambigüedad de doble camino: Como el no-terminal *Expr* desciende hasta *Identifier*, mantener `(Identifier | Expr)` creaba dos árboles de derivación válidos para una misma entrada. Se redujo a *Expr* para forzar un único camino de parseo.
- Vars
 - Vease el cambio de components. (Omisión de esta regla y uso de `+` o `*` en donde fuera necesario).
- Nota 1: Los caracteres `<` y `>` se escriben en Rascal como `\<` y `\>`. Se aplicaron estos en donde era necesario.